



CHRIST
(DEEMED TO BE UNIVERSITY)
BANGALORE · INDIA

Exercise-9 & 10: Develop a Responsive Personal Portfolio using Next.js

Problem Statement: Develop a personal portfolio website using Next.js and Tailwind CSS that presents your profile, skills, projects, education, achievements, and contact information. The application should demonstrate server-side rendering (SSR), client-side rendering (CSR), API-based data fetching, and mobile-first responsive design.

Mission

Christ University is a nurturing ground for an individual's holistic development to make effective contribution to the society in a dynamic environment

Vision

Excellence and Service

Core Values

Faith in God | Moral Uprightness
Love of Fellow Beings | Social Responsibility
Pursuit of Excellence

Step-1: Check Environment and Create Next.js Project

node -v

npm -v

Recommended: Node.js, npm, VS Code, Web Browser

Create Next.js Project using: `npx create-next-app@latest my-portfolio`

Would you like to use TypeScript? → No

Would you like to use ESLint? → Yes

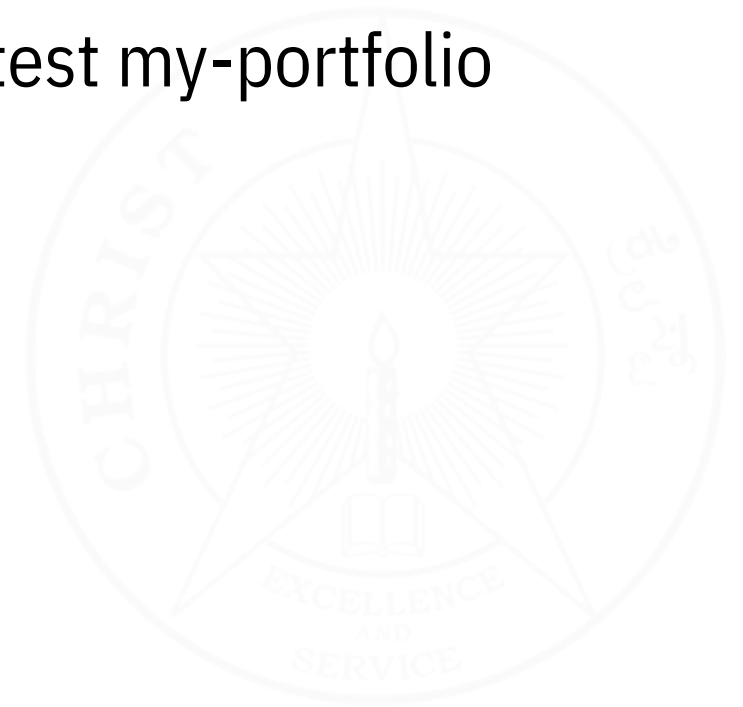
Would you like to use Tailwind CSS? → Yes

Would you like your code inside a src/ directory? → No

Would you like to use App Router? → Yes

Would you like to customize the import alias? → No

then: `cd my-portfolio`



Step 2: Run the Application and Understand the Project Structure

npm run dev

Open: <http://localhost:3000>

You should see the default Next.js page.

my-portfolio

```
├── app
│   ├── page.js
│   ├── layout.js
│   └── globals.css
├── public
├── package.json
├── next.config.js
└── node_modules
```

Important: app/page.js represents: /

Step 3: Create Portfolio Pages

Create the following folders:

```
app
├── page.js
├── about
│   └── page.js
├── projects
│   └── page.js
├── skills
│   └── page.js
├── achievements
│   └── page.js
└── contact
    └── page.js
```

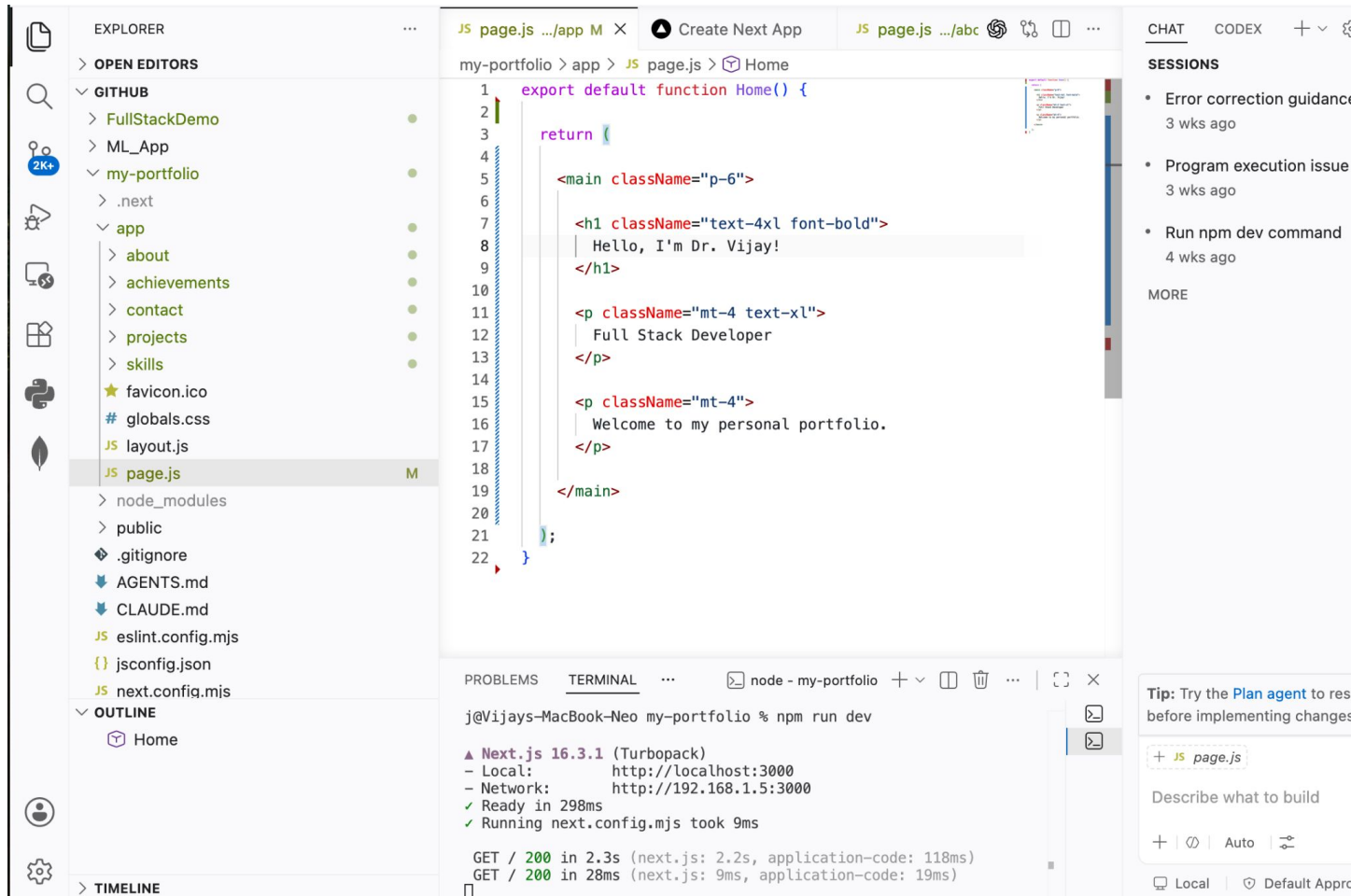
Now the routes automatically become:

```
/
/about
/projects
/skills
/achievements
/contact
```



Step 4: Create the Home Page

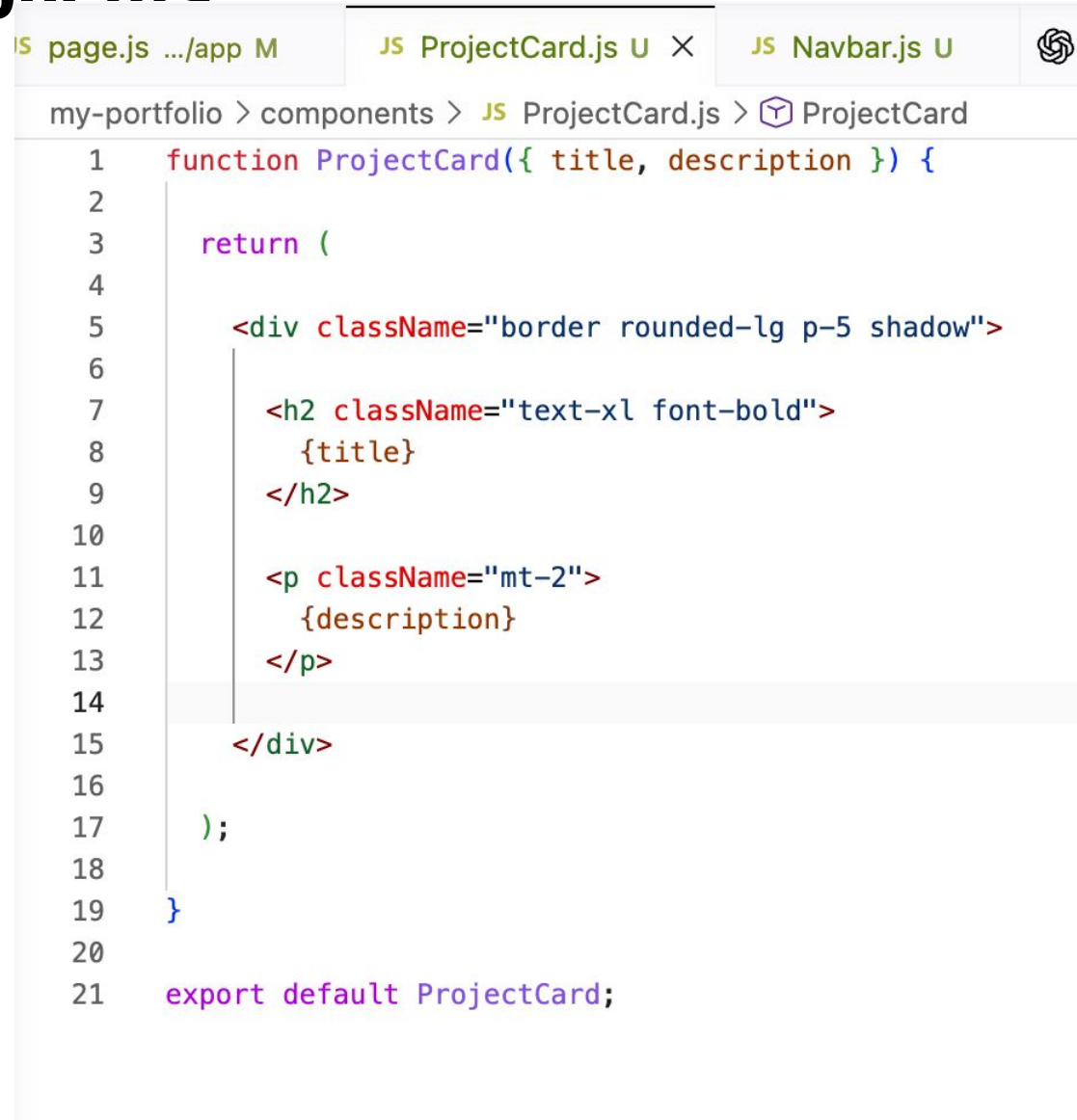
Open: `app/page.js` Create a simple introduction:



Step 5: Create Reusable Components

components

- └─ Navbar.js
- └─ ProjectCard.js
- └─ SkillCard.js
- └─ Footer.js



```
1  function ProjectCard({ title, description }) {
2
3      return (
4
5          <div className="border rounded-lg p-5 shadow">
6
7              <h2 className="text-xl font-bold">
8                  {title}
9              </h2>
10
11              <p className="mt-2">
12                  {description}
13              </p>
14
15          </div>
16
17      );
18
19  }
20
21  export default ProjectCard;
```

Step 6: Add Navigation

Create components/[Navbar.js](#):

Add the Navbar to layout.js.

```
JS page.js .../app M    JS ProjectCard.js U    JS Navbar.js U X    Create Next App
my-portfolio > components > JS Navbar.js > Navbar
3  export default function Navbar() {
6
7      <nav className="flex flex-wrap gap-5 p-5 border-b">
8
9          <Link href="/">Home</Link>
10
11         <Link href="/about">About</Link>
12
13         <Link href="/projects">Projects</Link>
14
15         <Link href="/skills">Skills</Link>
16
17         <Link href="/achievements">Achievements</Link>
18
19         <Link href="/contact">Contact</Link>
20
21     </nav>
22
23 };
24
25 }
```

Step 7: Create Projects Page

Open: app/projects/[page.js](#)

This page will demonstrate Server-Side Rendering / Server Components.

Use an API to fetch project-related data.

For demonstration, students can use: <https://jsonplaceholder.typicode.com/posts>



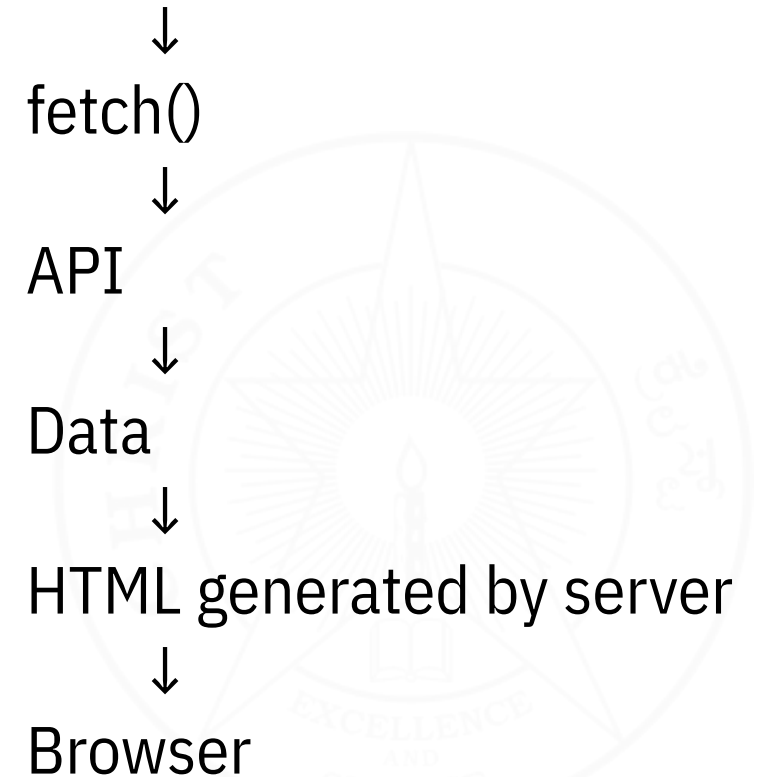
```

async function getProjects() {
  const response = await fetch(
    "https://jsonplaceholder.typicode.com/posts"
  );
  return response.json();
}

export default async function Projects() {
  const projects = await getProjects();
  return (
    <main className="p-6">
      <h1 className="text-3xl font-bold mb-6">
        My Projects
      </h1>
      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-5">
        {projects.slice(0, 6).map((project) => (
          <div
            key={project.id}
            className="border rounded-lg p-5 shadow"
          >
            <h2 className="font-bold">
              {project.title}
            </h2>
            <p className="mt-2">
              {project.body}
            </p>
          </div>
        ))}
      </div>
    </main>
  );
}

```

Next.js Server Component



Step 8: Create a Client Component

Create: components/[ProjectFilter.js](#)

The "use client" directive allows:

- useState
- Event handling
- Browser interaction
- Client-side rendering

```
1  "use client";
2  import { useState } from "react";
3  export default function ProjectFilter() {
4
5      const [category, setCategory] = useState("All");
6
7      return (
8
9          <div className="mb-6">
10
11              <select
12                  value={category}
13                  onChange={(e) => setCategory(e.target.value)}
14                  className="border p-3 rounded"
15              >
16
17                  <option>All</option>
18                  <option>Web</option>
19                  <option>AI</option>
20                  <option>Mobile</option>
21
22              </select>
23
24              <p className="mt-3">
25                  Selected Category: {category}
26              </p>
27
28          </div>
29      );
30 }
```

Step 9: Use the Client Component

In the Projects page: import ProjectFilter from "../components/ProjectFilter";

Then: <ProjectFilter />

SERVER COMPONENT

- Fetch data
- Render on server

CLIENT COMPONENT

- User interaction
- useState
- Browser



Step 10: Add Tailwind CSS

Students should use Tailwind classes rather than writing everything in CSS.

Example: `<div className="bg-white p-6 rounded-xl shadow-lg">`

Tailwind	Meaning
p-6, m-6	Padding, Margin
text-3xl, font-bold	font size, bold
rounded-lg	Rounded corners
shadow	Shadow
flex	Flexbox
grid	Grid

Step 11: Implement Mobile-First Design

Use:

```
<div className="
```

```
  grid
```

```
  grid-cols-1
```

```
  sm:grid-cols-2
```

```
  lg:grid-cols-3
```

```
  gap-6
```

```
">
```



Step 12: Add Dynamic Project Route

Create: `app/projects/[id]/page.js`

This produces:

- `/projects/1`
- `/projects/2`
- `/projects/3`



Step 13: Complete Portfolio Content

Validate

- Empty fields
- Invalid inputs
- Duplicate entries (if applicable)

Display appropriate success or error messages.

```
1 export default async function ProjectDetails({ params }) {  
2  
3   const { id } = await params;  
4  
5   const response = await fetch(  
6     `https://jsonplaceholder.typicode.com/posts/${id}`  
7   );  
8  
9   const project = await response.json();  
10  
11  return (  
12  
13    <main className="p-6">  
14  
15      <h1 className="text-3xl font-bold">  
16        {project.title}  
17      </h1>  
18  
19      <p className="mt-4">  
20        {project.body}  
21      </p>  
22  
23    </main>  
24  
25  );  
26  
27 }
```

Step 14: Complete Portfolio Content

Students now replace the demonstration content with their actual information.

Home

- Name
- Profile
- Career objective

About

- Education
- Short biography

Skills

- Programming languages
- Frameworks
- Tools

Projects

- Project name
- Description
- Technologies
- GitHub/demo link

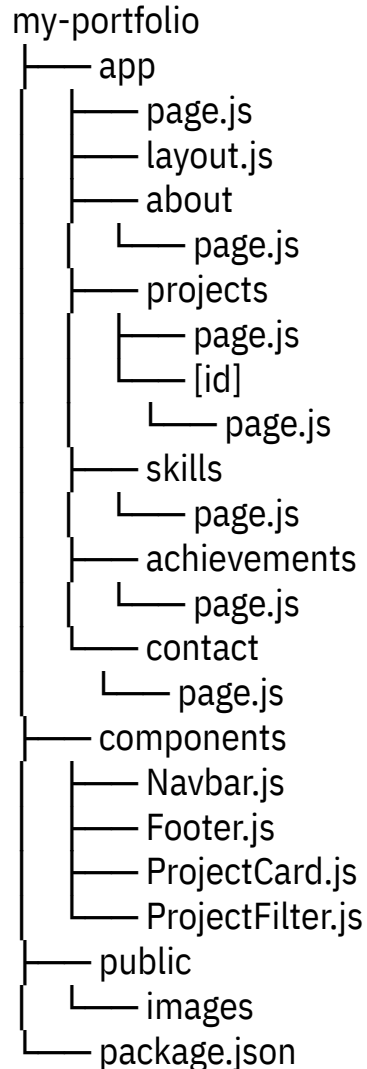
Achievements

- Certifications
- Awards

Contact

- Email
- LinkedIn
- GitHub
- Contact information

Step 15: Final Application Structure



1. Routing

/about

/projects

/projects/1

2. Server-Side Rendering

Show the Projects page fetching API data through a Server Component.

3. Client-Side Rendering

Show the interactive Project Filter using:
"use client" - useState()

4. API Data Fetching

fetch()

and where the data comes from.

5. Tailwind CSS

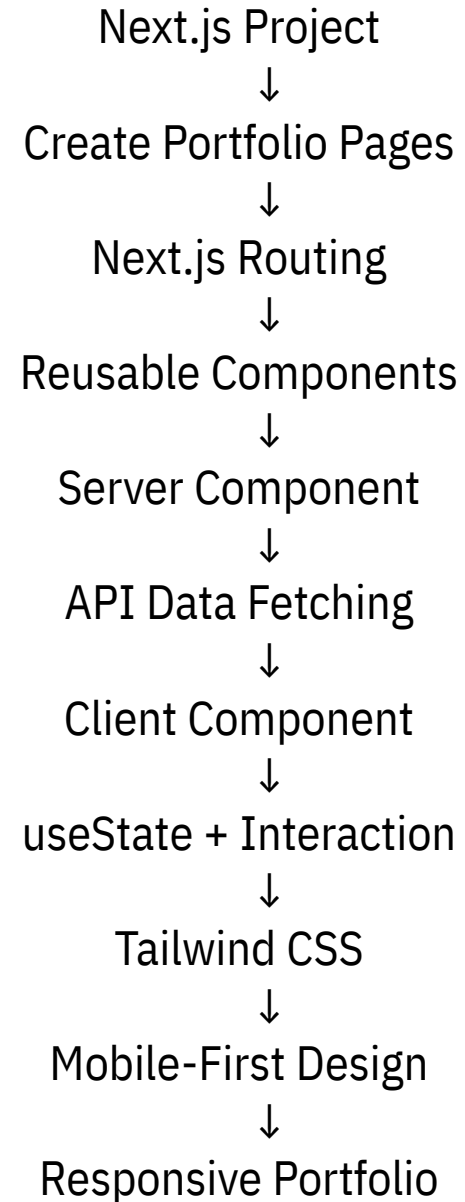
Show the Tailwind classes used for styling.

6. Mobile-First Design

Resize the browser and demonstrate:

Mobile → Tablet → Desktop

Final Workflow



Marking Rubrics

- Execution/ Implementation – 2M
- Formatting & Validation – 2M
- Concept Clarity/ Viva– 2M
- Complexity/ Self Learning/ On-Spot Task Completion – 2M
- Deployment – 2M

